

2. The reimbursement of sales staff's public funds consumption needs to be approved by the supervisor, manager and director. After approval, the financial department will make payment. The reimbursement process must be in the order of "supervisor -> Manager -> director". State pattern is useful with this scenario. Please write the code to implement the pattern.

Hint: reimbursement has four statuses: no signature, signed by the supervisor, signed by the supervisor and manager, and signed by the supervisor, manager and director. In each status, there are four operations to withdraw money from the financial department, request the signature of the supervisor, request the signature of the manager and request the signature of the director.

Client

```
package experiment20_2;

public class Client {

    public static void main(String[] args) {

        Reimbursement reimbursement = new
Reimbursement();

        reimbursement.requestSupervisorSignature();
        reimbursement.requestManagerSignature();
        reimbursement.requestDirectorSignature();
        reimbursement.withdrawMoney();

    }

}
```

DirectorSignedState

```
package experiment20_2;
```

```
class DirectorSignedState implements ReimbursementState{

    @Override

    public void withdrawMoney(Reimbursement
reimbursement) {

        System.out.println("Withdraw money successfully.");

    }

    @Override

    public void requestSupervisorSignature(Reimbursement
reimbursement) {

        System.out.println("Supervisor's signature is already
obtained.");

    }

    @Override

    public void requestManagerSignature(Reimbursement
reimbursement) {

        System.out.println("Manager's signature is already
obtained.");

    }

    @Override
```

```

    public void requestDirectorSignature(Reimbursement
reimbursement) {
        System.out.println("Director's signature is already
obtained.");
    }
}

```

ManagerSignedState

```

package experiment20_2;

class ManagerSignedState implements ReimbursementState{
    @Override
    public void withdrawMoney(Reimbursement
reimbursement) {
        System.out.println("Cannot withdraw money, require
signatures of Director.");
    }

    @Override
    public void requestSupervisorSignature(Reimbursement
reimbursement) {
        System.out.println("Supervisor's signature is already
obtained.");
    }
}

```

```

    }

    @Override
    public void requestManagerSignature(Reimbursement
reimbursement) {
        System.out.println("Manager's signature is already
obtained.");
    }

    @Override
    public void requestDirectorSignature(Reimbursement
reimbursement) {
        System.out.println("Requesting director's
signature...");
        reimbursement.setReimbursementState(new
DirectorSignedState());
    }
}

```

NoSignatureState

```

package experiment20_2;

class NoSignatureState implements ReimbursementState{

```

@Override

```
public void withdrawMoney(Reimbursement  
reimbursement) {  
    System.out.println("Cannot withdraw money, require  
signatures of Supervisor.");  
}
```

@Override

```
public void requestSupervisorSignature(Reimbursement  
reimbursement) {  
    System.out.println("Requesting supervisor's  
signature...");  
    reimbursement.setReimbursementState(new  
SupervisorSignedState());  
}
```

@Override

```
public void requestManagerSignature(Reimbursement  
reimbursement) {  
    System.out.println("Cannot request manager's  
signature. Supervisor's signature is required first.");  
}
```

```

@Override

public void requestDirectorSignature(Reimbursement
reimbursement) {

    System.out.println("Cannot request director's
signature. Supervisor's signature is required first.");

}

}

```

Reimbursement

```

package experiment20_2;

class Reimbursement {

    private ReimbursementState reimbursementState;

    public Reimbursement(){

        reimbursementState = new NoSignatureState();

    }

    public void setReimbursementState(ReimbursementState
reimbursementState){

        this.reimbursementState = reimbursementState;

    }

    public void withdrawMoney(){

        reimbursementState.withdrawMoney(this);

    }

}

```

```

    }

    public void requestSupervisorSignature(){

reimbursementState.requestSupervisorSignature(this);

    }

    public void requestManagerSignature(){

        reimbursementState.requestManagerSignature(this);

    }

    public void requestDirectorSignature(){

        reimbursementState.requestDirectorSignature(this);

    }

}

```

ReimbursementState

```

package experiment20_2;

interface ReimbursementState {

    void withdrawMoney(Reimbursement reimbursement);

    void requestSupervisorSignature(Reimbursement
reimbursement);

    void requestManagerSignature(Reimbursement
reimbursement);
}

```

```
    void requestDirectorSignature(Reimbursement  
reimbursement);  
}
```

SupervisorSignedState

```
package experiment20_2;  
  
class SupervisorSignedState implements  
ReimbursementState{  
    @Override  
    public void withdrawMoney(Reimbursement  
reimbursement) {  
        System.out.println("Cannot withdraw money, require  
signatures of Manager.");  
    }  
  
    @Override  
    public void requestSupervisorSignature(Reimbursement  
reimbursement) {  
        System.out.println("Supervisor's signature is already  
obtained.");  
    }  
}
```



```

@Override

    public void requestManagerSignature(Reimbursement
reimbursement) {

        System.out.println("Requesting manager's
signature...");

        reimbursement.setReimbursementState(new
ManagerSignedState());

    }

@Override

    public void requestDirectorSignature(Reimbursement
reimbursement) {

        System.out.println("Cannot request director's
signature. Manager's signature is required first.");

    }

}

```

Output

```

D:\JDK\bin\java.exe "-javaagent:D:\idea\Inte
Requesting supervisor's signature...
Requesting manager's signature...
Requesting director's signature...
Withdraw money successfully.

Process finished with exit code 0

```